

Task 1 – F1 Pit Strategy via SSL & Active Learning

Formula 1 (F1) is the highest class of international single seater auto racing. Each season consists of a series of races called Grands Prix held at circuits around the world. F1 victory is decided not only by driver skill, but by engineering, aerodynamics, and critical race strategy. One of the most consequential strategic decisions a team makes during a race is the pit stop.

A pit stop is when a car leaves the track, enters the pit lane, and pauses briefly so mechanics can change tires, make repairs, or adjust the car. Each stop costs roughly 20–30 seconds of race time.

There are four main pit stop strategies teams use:

- **Undercut:** The car pits slightly before the rival ahead, gets fresh tyres, laps faster, and rejoins in front once the rival pits. Requires closing the gap to the car ahead.
- **Overcut:** A strategic move where a driver remains on track longer than an immediate rival who has already pitted. By staying out, the driver exploits "clean air" to set fast lap times on older tyres, capitalizing on the rival's struggle to warm up fresh tyres or their entrapment in traffic. This allows the driver to either overtake a rival ahead or defend their position against a rival behind once they finally pit.
- **Standard:** A planned stop based purely on tyre life targets with no tactical intent to overtake or defend. The most common strategy
- **Emergency:** An unplanned or reactive stop triggered by tyre damage, a safety car, heavy rain, or sudden tyre failure. High urgency, no prior planning.

This task asks you to predict which of these four strategies was used for a given pit stop, using only the race and lap features available at the moment of the decision and to do so with very limited labeled data. You will engineer features from multiple raw CSV files, build supervised baselines, and then apply two semi-supervised learning techniques: Pseudo Labeling and Active Learning.

Dataset Description

You will use the [Formula 1 World Championship \(1950-2024\)](#) dataset from Kaggle, compiled from the Ergast Developer API.

Files you may need: races.csv, pit_stops.csv, lap_times.csv, results.csv, driver_standings.csv, constructor_standings.csv and constructors.csv.

After downloading and joining the files you will build a lap level feature matrix for all 2014–2024 pit stops, assign strategy labels, and then produce two CSV files via a random split:

- **labeled.csv:** A randomly sampled ~10% of all 2014–2024 pit stops with all engineered features AND the strategy label. This simulates the scarce human annotated data available.
- **unlabeled.csv:** The remaining ~90% of 2014–2024 pit stops with all engineered features but NO labels. Both files come from the same era and the same distribution.

1. Data Loading & Feature Engineering (25 Points)

Raw CSV files from the Ergast dataset are relational. Each file represents one table linked by integer IDs. Your first job is to join them and build a rich, lap level feature matrix where each row is one pit stop event.

1.1 Download and Inspect (3 pts)

Download and load CSVs with pandas, display shapes/head rows, and document missing values, data types, and key IDs.

1.2 Join the Tables (5 pts)

Filter to 2014–2024 seasons first, then merge the CSV files into a single DataFrame. Include at minimum:

- Pit stop lap number and stop duration in milliseconds
- Driver's grid position at race start and current race position on the pit lap
- Constructor (team) name, circuit name, and race year
- Driver and constructor championship points at the time of the stop

Hint: To get championship points before the race, join with the previous raceId for each driver/constructor. You may use the round column and year ordering to identify the prior race.

1.3 Engineer Lap Level Features (17 pts)

Part A: Required Features (7 pts)

For each pit stop, compute the following features from the provided dataset:

- **lap_time_delta (ms):** Driver's lap time on the pit lap minus their personal best in that race. A large positive value signals tyre degradation.
- **laps_since_last_pit:** Laps elapsed since the driver last pitted. Proxy for tyre age.
- **laps_remaining:** Laps remaining in the race after this stop.
- **approx_gap_ahead (s):** Gap in seconds to the car immediately ahead (Guideline: Calculate the cumulative race time for each driver up to the pit lap from lap_times, then subtract the cumulative time of the driver in the immediately higher position).
- **approx_gap_behind (s):** Gap in seconds to the car immediately behind (approximate from positions and lap times using a corresponding cumulative time difference).
- **safety_car_likely (0/1):** Flag 1 if multiple drivers on the same lap are unusually slow (safety car proxy).

Part B: Open Ended Feature Engineering (10 pts Bonus)

Engineer at least 2 additional features that you believe capture tyre degradation, competitive pressure, or strategic context. You must justify each feature with an F1 domain argument and show its discriminative power with a visualization

2. Strategy Labeling & Dataset Split (5 Points)

2.1 Define Labeling Rules (3 pts)

Instead of using predefined labels, you must design and implement your own rule based heuristics to assign one of four strategy labels to each 2014–2024 pit stop: Standard, Undercut, Overcut, or Emergency.

Use the features engineered in Section 1.3 (or develop your own customized features) to formulate these conditions. To ensure your rules reflect real world Formula 1 racing dynamics, your final annotated dataset should roughly approximate the following logical class distributions:

Standard (~65% – 75%): The baseline scenario. The vast majority of pit stops are scheduled, routine tyre changes.

Undercut (~12% – 18%): Tactical stops triggered when a driver is close to the car ahead, leveraging fresh tyres to pass them during the pit cycle.

Emergency (~8% – 12%): Highly disruptive, unplanned stops. This includes pitting under safety car/VSC conditions, dealing with sudden punctures, or managing extreme unexpected tyre degradation.

Overcut (~4% – 8%): Strategic extension of a stint to gain an advantage. This is generally less common in the modern Pirelli era and highly track dependent.

2.2 Random Split into labeled.csv and unlabeled.csv (2 pts)

1. Apply `train_test_split` with `test_size=0.90` to split all labeled pit stops into a ~10% labeled set and a ~90% unlabeled pool and `stratify=y` (where `y` is the strategy label column) to ensure proportional class distribution in both files.

2. Save the ~10% portion (with strategy column) as `labeled.csv`.

3. Save the ~90% portion (strategy column dropped) as `unlabeled.csv`.

4. Verify: print class counts in `labeled.csv`, shapes of both files, and 3 sample rows from each.

5. Confirm the split is fair: compare class proportions in `labeled.csv` vs the full pool. They should be similar because of stratification.

3. Exploratory Data Analysis (12 Points)

3.1 Class Distribution (3 pts)

- Plot a bar chart of strategy class counts in `labeled.csv` with percentage labels.
- Plot a side by side bar chart comparing class proportions in `labeled.csv` vs the full 2014-2024 pool. Confirm the stratified split preserves the distribution.

3.2 Feature Distributions by Strategy (4 pts)

- For each of the four key features (`laps_since_last_pit`, `stop_duration_ms`, `lap_time_delta`, `laps_remaining`), plot overlapping histograms coloured by strategy class.
- Which features most clearly separate classes? Explain the F1 reasoning behind each.
- Plot a correlation heatmap of all numeric features.

3.3 Split Quality Check (2 pts)

Since labeled and unlabeled data come from the same era and distribution, the feature distributions should be statistically similar.

- For at least 4 features, plot side by side histograms comparing labeled vs unlabeled.
- Run a Kolmogorov-Smirnov (KS) test for each feature. You should observe $p > 0.05$ (no significant difference). Explain why this matters.

(Note: The KS test is designed for continuous variables; it is not suitable for binary features such as `safety_car_likely`).

3.4 Constructor & Circuit Patterns (3 pts)

- Which constructors use the most Undercut strategies? Plot a grouped bar chart.
- Are Emergency stops more common at specific circuits? Visualise with a heatmap.
- At which circuits is the `approx_gap_ahead` smallest on average? Why might this be?

4. Supervised Learning Baseline (15 Points)

Before using semi-supervised methods, establish how well a model trained only on `labeled.csv` performs. This baseline is your reference point for measuring SSL improvement.

4.1 Preprocessing (3 pts)

Drop `safety_car_likely` and `stop_duration_ms` to prevent rule reverse engineering. Encode categorical features (`constructor_name`, `circuit_name`) with justification, scale numeric data via `StandardScaler`, stratify split `labeled.csv` into train/validation/test (80/10/10%), and address class imbalance using `class_weight='balanced'` (explain its necessity).

4.2 Train Multiple Classifiers (6 pts)

Train and compare at least three classifiers: Logistic Regression, Random Forest Classifier and Gradient Boosting Classifier.

To prevent complete rule memorization and ensure a smooth learning curve for the subsequent SSL and Active Learning tasks, apply regularization to the tree-based models (e.g., limit `max_depth` or increase `min_samples_leaf`).

Use the validation Macro F1-score to select the best model. Report all three in a table.

4.3 Evaluate on Test Set (6 pts)

For your best classifier, compute and report:

- Accuracy, Macro Precision, Macro Recall, Macro F1-score
- Per class F1-score and confusion matrix heatmap
- Feature importance plot. Which features drive predictions most?

5. Semi-Supervised Learning: Pseudo Labeling (20 Points)

Pseudo labeling is an iterative SSL technique where a model trained on labeled data predicts labels for unlabeled data. High-confidence predictions (above a threshold) are added to the training pool as pseudo-labels, and the model is iteratively retrained. Key concepts:

- **Confirmation Bias:** If the model makes systematic errors, those wrong labels get added and reinforced over rounds. Careful threshold selection is critical.
- **Confidence Threshold:** Controls the quality vs. quantity trade-off. Too low → noisy additions. Too high → too few samples added.

5.1 Implementation (13 pts)

1. Train the best baseline classifier on the training portion of labeled.csv (the %80 split from section 4.1)
2. Use `predict_proba()` on the unlabeled pool. Confidence = max probability across all classes.
3. Select samples where confidence $\geq$ threshold (start with 0.75) and assign the `argmax` class as a pseudo label.
4. Combine the original labeled training data with the newly pseudo-labeled samples.
5. Retrain the model on the expanded set and evaluate on the held-out test set.
6. Repeat steps 2–5 for at least 5 rounds or until fewer than 50 new samples are added.
7. Also run the full experiment with thresholds 0.60 and 0.85. Compare all three.

5.2 Analysis (7 pts)

- Plot how many pseudo labels were added per round, broken down by class. Does the model confidently pseudo-label certain strategies more than others?
- Plot the test Macro F1-score after each round for all three threshold values on the same figure.

- At which round and threshold do you observe the best improvement? Where does it plateau?
- Set threshold to 0.55. What happens to the learning curve? Explain in terms of confirmation bias.

6. Active Learning (25 Points)

Active Learning interactively queries an expert to label the most informative samples, mimicking an F1 analyst reviewing limited ambiguous pit stops. Three query strategies identify uncertainty: Least Confidence selects the lowest predicted top class probability; Margin targets the smallest difference between the top two probabilities; and Entropy chooses the highest Shannon entropy across all classes.

6.1 Implementation (15 pts)

1. Start with the initial small labeled training set and train your best baseline classifier.
2. For each active learning round, compute uncertainty scores for all samples in the unlabeled pool using your chosen query strategy.
3. Select the top k most uncertain samples (use $k = 20$ per round).
4. Simulate human annotation: use the same rule-based labeling function from section 2 to assign labels to the selected k samples. This simulates querying a domain expert.
5. Add the newly labeled samples to the training set and retrain the model.
6. Record the model's test set Macro F1-score after each round.
7. Run for at least 8 rounds. Implement and compare all three query strategies.

6.2 Analysis (10 pts)

- Plot model performance (test Macro F1) vs. total labeled samples for all three query strategies on one figure. Add a horizontal baseline line.
- Which strategy reaches baseline performance fastest? Report the exact round and label count where each first matches the baseline.
- Inspect which classes were queried most in round 1. Are Emergency stops overrepresented? Why does this make sense?

- Compare Active Learning vs. Pseudo Labeling: AL uses $k=20$ human labels per round; PL uses up to 800 model generated labels per round. Which approach reaches the same F1 with fewer total labeled samples?
- Discuss the trade-off between annotation cost (manual labeling) and data volume (pseudo labels). When would each method be preferred in a real F1 team scenario?

7. Comparative Analysis (8 Points)

7.1 Summary Table (2 pts)

Create a summary table:

- Rows: Baseline | Pseudo Labeling Round 1 ... Final | Active Learning (each strategy, final round)
- Columns: Total Labeled Samples, Accuracy, Macro Precision, Macro Recall, Macro F1

7.2 ROC Curves & AUC (4 pts)

- For the Baseline, best Pseudo Labeling model, and best Active Learning model, plot one vs. the rest ROC curves for all four strategy classes on a 1×3 subplot grid.
- Report per class AUC and mean AUC for each method.
- Which class has the lowest AUC across all methods? Relate this to its label count.

7.3 Learning Curves (2 pts)

- Plot Macro F1 vs. total labeled samples on one figure showing Pseudo Labeling (best threshold), all three AL strategies, and the Baseline line.
- At what labeled set size does each method stop providing meaningful gains?

Final Discussion

Wrap up by thinking through these final questions. Make sure you understand them, but you don't need to put the answers in your report.

1. Which method Pseudo Labeling or Active Learning performed better, and why? Consider the effect of label scarcity and class imbalance.

2. In this task, labeled and unlabeled data come from the same distribution. How do you expect results would change if the unlabeled data came from a different era with different tyre rules and scoring systems?

3. In a real F1 team with a live data pipeline, which method would you deploy, and what safeguards would you put in place to avoid confirmation bias?